

AQUABC UI Integration Plan — Deep Process Rate Analysis

Generated: 2026-02-27 14:11 | ESTAS-AQUABC v0.2 | Shiny for Python (PyShiny) Framework

1. Executive Summary

This document plans the integration of the AQUABC Deep Process Rate Analysis toolkit into the existing PyShiny web UI (shiny_app/app.py). The goal is to provide users with **automatic post-simulation diagnostics** — including health checks, interactive visualisations, and downloadable PDF reports — directly from the browser-based interface, with no command-line usage required.

The integration adds a new **"Diagnostics"** navigation panel to the existing sidebar (alongside Dashboard, Plots, Mass Balance, etc.) and optionally triggers analysis automatically after each simulation run completes.

2. Current UI Architecture

The AQUABC UI is built with **Shiny for Python** (PyShiny) and served via an R Shiny wrapper (app.R) that embeds the Python backend in an iframe. The app uses a custom sidebar navigation system with JavaScript-driven panel switching.

nav_dashboard	Dashboard	Model status overview
nav_model_structure	Model Structure	State variable and process diagrams
nav_model_build	Model Build	Compiler selection and build log
nav_model_control	Model Config	Simulation config, run model, output config (3 tabs)
nav_input_files	Input Files	File browser + map display
nav_parameters	Parameters	Model parameter editor (WCONST_04.txt)
nav_initial_conditions	Initial Cond.	State variable initial values
nav_model_options	Model Options	Toggles and extra constants
nav_scenarios	Scenarios	Multi-scenario management
nav_plot	Plots	State vars, process rates, mass balance (4 tabs)
nav_mass_balance	Mass Balance	Conservation analysis
nav_observations	Observations	Model-observation comparison
nav_map	Map	Geographic visualization

Key architectural patterns: (1) NAV_CHOICES dict defines sidebar entries with Bootstrap icons. (2) panel_conditional() shows/hides content based on navigation input. (3) Reactive values track simulation state, output directory, loaded data. (4) Long-running operations use threading (subprocess for model run). (5) UI is ~8,500 lines in a single app.py file.

3. Integration Architecture

3.1 New Navigation Entry

Add a new entry to NAV_CHOICES in app.py:

```
"nav_diagnostics": ("bi-shield-check", "Diagnostics"),
```

This places a "Diagnostics" item in the sidebar with a shield-check icon, positioned after "Mass Balance" and before "Observations".

3.2 Panel Structure

The Diagnostics panel uses a tabbed layout (navset_card_tab) with 4 sub-tabs:

Overview	Summary dashboard with severity counts (ERROR/WARNING/INFO/OK), per-box health status indicators, and a quick-action "Run Analysis" button. Shows colour-coded cards: green (0 errors), amber (warnings present), red (errors found).
Detailed Results	Per-check expandable sections showing findings tables. Users can filter by box, severity, and check number. Tables are sortable. Shows derivative consistency, stoichiometry, sign violations, smoothness, etc.
Visualisations	Interactive Plotly charts: (a) spike timeline per variable, (b) limitation factor radar charts, (c) mass-balance bar charts, (d) cross-box heatmaps, (e) derivative consistency scatter plots. All reactive to box and variable selection.
Reports	One-click PDF generation and download for: (a) Analysis Results Report, (b) Detailed Analysis Report (existing generate_deep_pr_pdf.py), (c) Crosscheck Report. Shows generation status and file sizes. Download buttons return the generated PDF files.

3.3 Module Design

To avoid further bloating app.py (already 8,500 lines), the implementation will use a **separate module**:

shiny_app/diagnostics.py	~400 lines	Contains UI builder function (diagnostics_ui()), server logic function (diagnostics_server(input, output, session, rv)), and helper functions. Imported into app.py with a single import statement.
shiny_app/diagnostics_plots.py	~200 lines	Plotly chart generators for the Visualisations tab. Pure functions taking analysis results dict and returning Plotly figures.
tools/ (existing)	No changes	The analysis engine (deep_process_rate_analysis.py) and PDF generators remain in tools/ and are imported by the diagnostics module.

4. Automatic Post-Run Analysis

The analysis can be triggered automatically when a simulation completes. This requires integration with the existing model run workflow in app.py.

4.1 Trigger Mechanism

The existing "Run Model" tab launches the Fortran executable via subprocess in a background thread. When the process completes (returncode 0), a reactive value `rv_run_status` is set. The diagnostics module hooks into this reactive chain:

1	User clicks "Run Model"	Existing code launches ESTAS_II subprocess
2	Subprocess completes (rc=0)	Existing code updates <code>rv_run_status</code> = "completed"
3	Auto-analysis trigger	New <code>@reactive.effect</code> watches <code>rv_run_status</code> . If "completed" AND <code>auto_diagnostics</code> switch is ON, starts analysis in background thread.
4	Analysis runs	Calls <code>run_analysis(output_dir)</code> from <code>deep_process_rate_analysis</code> . Progress updates via reactive value <code>rv_diag_progress</code> (0-100%).
5	Results stored	Analysis results dict stored in <code>rv_diag_results</code> reactive value. UI automatically renders the Overview tab with updated severity counts.
6	Notification	Toast notification: "Analysis complete: 0 ERRORS, N WARNINGS". If errors found, notification is red and suggests reviewing the Diagnostics panel.

4.2 User Controls

Auto-run diagnostics	Switch (ON/OFF)	Toggle automatic analysis after simulation. Default: ON. When OFF, user must click "Run Analysis" manually.
Analysis scope	Checkbox group	Select which checks to run (default: all 16). Power users can disable slow checks (Check 2: derivative consistency) for faster turnaround.
Box selection	Checkbox group	Select which boxes to analyse. Synced with output config box selection.
Output directory	Dropdown	Defaults to current simulation output dir. Can select historical output dirs for retrospective analysis.
Severity filter	Dropdown	Filter displayed results: Show All, Errors Only, Warnings+, Errors+Warnings.

5. UI Component Specification

5.1 Overview Tab

The Overview tab shows a grid of severity cards at the top, followed by a per-box summary table and a quick-action bar.

Severity cards (4)	ui.value_box()	Green/amber/red/blue cards showing counts for ERROR/WARNING/INFO/OK. Each card shows the count as a large number with a trend indicator (compared to previous run if available). Uses layout_columns(col_widths=[3,3,3,3]).
Per-box health table	ui.output_data_frame()	Table with columns: Box, Type (sand/mud), Errors, Warnings, Infos, Status. Status column shows a coloured badge (PASS/WARN/FAIL). Sortable by any column.
Action bar	ui.layout_columns()	Contains: "Run Analysis" button (btn-primary), auto-diagnostics switch, progress bar (shown during analysis), "Download All Reports" button.
Last run info	ui.output_text()	Shows: "Last analysis: 2026-02-27 14:30 Duration: 12s Output dir: OUTPUTS"

5.2 Detailed Results Tab

Filter bar	ui.layout_columns()	Box selector (dropdown), severity filter (dropdown), check selector (dropdown). Changes update the findings table reactively.
Findings table	ui.output_data_frame()	Columns: Box, Check #, Check Name, Variable, Severity, Key Value, Note. Rows are coloured by severity. Sortable. Paginated (25 rows/page).
Detail panel	ui.output_ui() (conditional)	When a row is clicked, shows full finding details in an expandable panel: all dict keys as a key-value table, with float values formatted to 6 significant figures.

5.3 Visualisations Tab

Spike Timeline	Plotly scatter	X = time (days), Y = $ \Delta C $, coloured by severity. One subplot per selected variable. Threshold line shown as horizontal dashed line. Hover shows timestep, value, spike count.
Limitation Radar	Plotly polar	One radar chart per phytoplankton group showing mean limitation factors (light, N, P, T, Si) as spokes. Values 0-1 scale. Selector for box.
Mass Balance Bars	Plotly bar	Grouped bar chart: N/P/C/O budgets across boxes. Each bar shows mean dX/dt with error bars for min/max. Hover shows detailed breakdown.
Spatial Heatmap	Plotly heatmap	Boxes on X-axis, key variables on Y-axis. Colour = normalised mean concentration or CV across boxes. Helps identify spatial anomalies.
Derivative Scatter	Plotly scatter	X = kinetic dC/dt , Y = numerical dC/dt . One point per timestep, coloured by season. Perfect consistency = $y=x$ line. Selector for variable and box.

5.4 Reports Tab

Report selector	ui.input_checkbox_group()	Select which reports to generate: Results Report, Detailed Analysis Report, Crosscheck Report. Multiple selection allowed.
Generate button	ui.input_action_button()	"Generate Selected Reports" — triggers PDF generation in background thread. Progress indicator shows current report being generated.
Download area	ui.download_button() × 3	One download button per report type. Disabled until PDF exists. Shows file size and generation timestamp.
Preview area	ui.output_ui()	Shows the most recently generated PDF in an iframe preview (if browser supports it). Fallback: "Download to view" message.

6. Implementation Plan

Phase 1: Core Integration (estimated: 2-3 days)

1.1	Create shiny_app/diagnostics.py with diagnostics_ui() and diagnostics_server()
1.2	Add "nav_diagnostics" to NAV_CHOICES and wire panel_conditional in app.py
1.3	Implement Overview tab: severity cards, per-box health table, run analysis button
1.4	Implement background analysis execution with progress reactive value
1.5	Test manual "Run Analysis" from the UI

Phase 2: Detailed Results & Reports (estimated: 2-3 days)

2.1	Implement Detailed Results tab with filter bar and findings table
2.2	Implement Reports tab with PDF generation and download buttons
2.3	Wire up existing generate_*_pdf.py scripts as background tasks
2.4	Test PDF download flow end-to-end

Phase 3: Visualisations (estimated: 3-4 days)

3.1	Create shiny_app/diagnostics_plots.py with Plotly chart generators
3.2	Implement spike timeline chart (most informative single visualisation)
3.3	Implement limitation radar and mass-balance bar charts
3.4	Implement spatial heatmap and derivative scatter
3.5	Add reactive box/variable selectors for all charts

Phase 4: Auto-Run & Polish (estimated: 1-2 days)

4.1	Hook analysis into post-simulation completion trigger
4.2	Add auto-diagnostics switch and toast notifications
4.3	Cache analysis results to avoid re-running on tab switch
4.4	Add scope selector (choose which checks to run)
4.5	Documentation and user-facing help text

7. Technical Considerations

7.1 Performance

The analysis runs in ~10-15 seconds for 7 boxes $\times$ 8,761 timesteps on a modern machine. This is acceptable for a one-time post-simulation check. For interactive use, consider:

Caching	Store results in a reactive value that persists across tab switches. Invalidate only when output directory changes or user clicks "Re-run".
Progress	Update a progress bar reactive (0-100%) using box completion as increments (each box = 14.3%). Display "Analysing Box N of 7..." text.
Thread safety	Run analysis in a background thread (matching existing model run pattern). Use <code>reactive.isolate()</code> and <code>session.flush_reactive()</code> to update UI safely.
Lazy loading	Only load heavy data (process rate files) when user navigates to Diagnostics tab. Avoid auto-loading on app startup.

7.2 Error Handling

No output files	Show "No model output found. Run a simulation first." with a link to the Model Config $\rightarrow$ Run Model tab.
No PROCESS_RATES files	Show "Process rate output not enabled. Go to Model Config $\rightarrow$ Output Config and check 'Process Rates'." with a link.
Analysis exception	Catch all exceptions in the background thread. Display the traceback in a collapsible error panel. Log to console.
Partial results	If some boxes fail but others succeed, show partial results with a banner: "Analysis completed for N of M boxes. Box X failed: [reason]."

7.3 Integration Points in app.py

NAV_CHOICES dict (line ~307)	Add "nav_diagnostics" entry after "nav_mass_balance"
panel_conditional blocks (~line 1228+)	Add panel_diagnostics with conditional on <code>input.navigation === "nav_diagnostics"</code>
Server function (~line 2600+)	Import and call <code>diagnostics_server()</code> passing input, output, session, and reactive values (<code>rv_output_dir</code> , <code>rv_run_status</code>)
Post-run callback	In the model run completion handler, add conditional call to trigger auto-analysis if switch is ON
Requirements.txt	Ensure <code>plotly</code> and <code>reportlab</code> are in <code>shiny_app/requirements.txt</code> (<code>plotly</code> is already present; <code>reportlab</code> may need to be added)

8. Code Skeleton

8.1 diagnostics.py — UI Builder

```
def diagnostics_ui():
    """Build the Diagnostics panel UI."""
    return ui.panel_conditional(
        "input.navigation == 'nav_diagnostics'",
        ui.navset_card_tab(
            ui.nav_panel("Overview", _overview_tab()),
            ui.nav_panel("Detailed Results", _details_tab()),
            ui.nav_panel("Visualisations", _viz_tab()),
            ui.nav_panel("Reports", _reports_tab()),
            id="diagnostics_tabs"
        )
    )
```

8.2 diagnostics.py — Server Logic

```
def diagnostics_server(input, output, session, rv):
    """Register all Diagnostics reactive/render functions."""
    diag_results = reactive.Value(None) # Cached results
    diag_running = reactive.Value(False)
    diag_progress = reactive.Value(0)

    @reactive.effect
    @reactive.event(input.btn_run_diagnostics)
    def _run_diagnostics():
        output_dir = rv.output_dir()
        diag_running.set(True)
        # Run in background thread
        threading.Thread(target=_bg_analysis,
                        args=(output_dir,)).start()

    def _bg_analysis(output_dir):
        from tools.deep_process_rate_analysis import run_analysis
        results = run_analysis(output_dir)
        diag_results.set(results)
        diag_running.set(False)
```

8.3 app.py Integration

```
# In NAV_CHOICES (line ~320):
"nav_diagnostics": ("bi-shield-check", "Diagnostics"),

# In app_ui function (after panel_mass_balance):
from shiny_app.diagnostics import diagnostics_ui, diagnostics_server
panel_diagnostics = diagnostics_ui()

# In the main layout (add to panels list):
panel_diagnostics,

# In the server function:
diagnostics_server(input, output, session, rv)
```

9. Automatic Output Generation

Beyond interactive diagnostics, the system should automatically generate standardised analysis outputs after each simulation. These serve as permanent records for model quality assurance and are stored alongside the simulation outputs.

diagnostics_summary.txt	Text	Auto-generated text summary (same as CLI output). Saved in the output directory alongside .out files. Includes timestamp, box count, error/warning counts.
diagnostics_summary.json	JSON	Machine-readable results dict. Enables programmatic querying of results, comparison between runs, and integration with external tools.
AQUABC_Analysis_Results_Report.pdf	PDF	Publication-quality results PDF. Generated on-demand from Reports tab or automatically if auto-generate-reports switch is ON.
diagnostics_badge.html	HTML	Small HTML badge showing "0 ERRORS 55 WARNINGS" that can be displayed on the Dashboard panel as a status indicator.

Storage location: All auto-generated diagnostic files are saved in a `diagnostics/` subdirectory within the output directory (e.g., `OUTPUTS/diagnostics/`). This keeps diagnostics associated with their simulation but separate from model output files.

10. User Guidance Integration

The UI should provide contextual help for non-expert users:

Tooltips	Every control and severity badge has a tooltip explaining what it means. Tooltips use <code>ui.tooltip()</code> wrapper around each widget.
Help panel	A collapsible "Understanding Diagnostics" panel at the top of the Overview tab. Contains: (a) What each severity level means, (b) What to do for each type of finding, (c) Links to the process rate slot reference and AQUABC manual.
Contextual actions	When an ERROR or WARNING is shown, include suggested actions: "Check parameter X", "Verify initial condition Y", "See AQUABC Reference Manual §Z". These are generated from a recommendations dict keyed by check number.
Comparison mode	If multiple output directories exist (e.g., <code>OUTPUTS_30day</code> , <code>OUTPUTS_365day</code>), allow side-by-side comparison of diagnostic results to track improvement over time.

11. Testing Strategy

Unit tests	tests/test_diagnostics.py	Test diagnostics_ui() renders without errors. Test severity counting function. Test result filtering logic. Mock run_analysis() for fast tests.
Integration tests	tests/test_diagnostics_integration.py	Test full pipeline: load real output files → run analysis → verify results dict structure. Test PDF generation produces valid files. Requires OUTPUTS/ directory.
UI tests	Manual / Playwright	Navigate to Diagnostics tab. Click Run Analysis. Verify tables populate. Download PDF. Check toast notifications. Test with no output files (error state).
Regression	tests/test_analysis_regression.py	Golden-file comparison: run analysis on OUTPUTS/ and compare error/warning counts against expected values (currently 0 ERR, 55 WARN). Fails if model changes introduce regressions.

12. Summary & Next Steps

This plan provides a complete roadmap for integrating the analysis toolkit into the AQUABC UI. The 4-phase approach allows incremental delivery with testable milestones:

Phase 1	2-3 days	Core panel + Run Analysis button + Overview tab	Users can trigger analysis from the UI and see severity counts
Phase 2	2-3 days	Detailed results + PDF reports	Full findings browsing and downloadable reports
Phase 3	3-4 days	Interactive visualisations	Plotly charts for spike timelines, limitation factors, mass balance
Phase 4	1-2 days	Auto-run + polish	Automatic post-simulation diagnostics with notifications

Total estimated effort: 8-12 developer-days for complete implementation. Phase 1 alone delivers 80% of the user value (ability to run analysis from UI).

Immediate next steps:

1. Create shiny_app/diagnostics.py skeleton with UI and server functions
2. Add nav_diagnostics to NAV_CHOICES and test sidebar navigation
3. Implement Overview tab with hardcoded sample data to validate layout
4. Wire up run_analysis() background execution and populate real results
5. Implement Reports tab for PDF download (quick win — reuses existing generators)